ASP.NET Core Request Processing Pipeline

Add to ASP.NET Core Tutorial Bookmarks

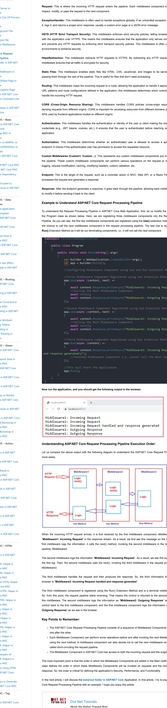
ASP.NET Core Request Processing Pipeline

In this article, I will discuss the **ASP.NET Core Request Processing Pipeline** with Examples. Please read our previous article before proceeding to this article where we discussed **Middleware Components in ASP.NET Core** Applications with Examples.

What is the ASP.NET Core Request Processing Pipeline?

The ASP.NET Core request processing pipeline is a series of middleware components that handle incoming HTTP requests and responses in an ASP.NET Core Web application. Each middleware component is responsible for a specific task, such as authentication, routing, logging, setting encryption and decryption, response generation, etc. The pipeline is configured in the Program class or a dedicated Startup class.

The following diagram shows the complete request processing pipeline for ASP.NET Core MVC, Web API, and Razor Pages applications.



The above diagram shows the ASP.NET Core Request Processing Pipeline and the sequence in which middleware components process incoming requests and outgoing responses. Each middleware component in ASP.NET Core application, can call another or pass the Request Processing Pipeline.

Request: This is where the incoming HTTP request enters the pipeline. Each middleware component in the pipeline can inspect, modify, or pass the request to the next middleware component.

ExceptionHandler: This middleware is often used to handle exceptions globally. If an unhandled exception occurs it catches the exception and returns a generic error response, such as a 500 Internal Server Error.

HTTS (HTTP Strict Transport Security): This middleware enforces strict security policies, telling browsers to only interact with the server using HTTPS. This helps protect against man-in-the-middle attacks. It adds headers to the HTTP response and ensures any HTTP requests by enforcing strict transport security policies. This middleware is often used in production environments to enhance security.

HttpRedirection: This middleware redirects HTTP requests to HTTPS. By redirecting any HTTP requests to HTTPS, this middleware ensures that all traffic is encrypted.

Static Files: This middleware serves static files like HTML, CSS, JavaScript, and images to a server directory without passing through the rest of the pipeline. This is ideal for files that don't need processing.

Routing: This middleware maps the incoming request to a route, allowing the application to handle it appropriately. Based on the URL and HTTP method, it defines the application. This middleware determines which controller and action will handle the incoming HTTP request.

CORS (Cross-Origin Resource Sharing): This middleware handles CORS policies (cross-origin requests), allowing or restricting resources from a web page to be requested from another domain. It adds the necessary headers to the HTTP response to enforce CORS policies.

Authentication: This middleware handles the authentication of a user. It checks if a user is claiming the request by validating credentials (e.g., JWT tokens, cookies, etc.). It ensures that the user is authenticated before reaching certain parts of the application.

Authorization: This middleware determines whether an authenticated user is allowed to access a specific resource or endpoint. It checks the authenticated user has permissions to access the requested resource.

Custom Middleware (Custom1, Custom2...): This represents any custom middleware components you may have added to the pipeline. These middleware components can perform various tasks, such as logging the request or response, adding headers, logging exceptions, caching, modifying request or response data, etc.

Endpoint: This is the target of the request, such as a controller action in MVC, Web API, or a Razor Page. The endpoint processes the request and generates a response.

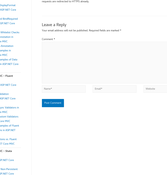
Response: After the endpoint generates a response, it goes back through the pipeline, allowing each middleware component to modify it before sending it back to the client.

Example to Understand ASP.NET Core Request Processing Pipeline

To understand the Request Processing Pipeline in ASP.NET Core Web Application, first let us create the Razor Pages Web Application. Please create an ASP.NET Core Web Application with the name **RequestPipelineDemo**. Select the Razor Pages template and uncheck the Configure for HTTPS and Enable Razor runtime compilation options. We will use the Razor Pages for checking the Request Processing Pipeline. The last step is to implement two custom middleware components in the request processing pipeline. The last step is registered using the Run extension method. Modify the Program.cs file as follows.

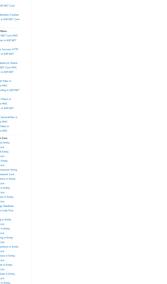
```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
//Configuring Middleware Component using Use and Run Extension Method
app.Use(async (context, next) =>
{
    //Write the message to the console
    Console.WriteLine("MyMiddleware1: Incoming Request");
    //Call the next middleware component
    var nextMessage = "MyMiddleware2: Incoming Response";
    Console.WriteLine("MyMiddleware1: Calling MyMiddleware2");
    //Write the message to the response
    await context.Response.WriteAsync("MyMiddleware1: Incoming Response\n");
    //Call the next middleware component
    await next();
    //Write the message to the response
    nextMessage = "MyMiddleware1: Outgoing Response";
    Console.WriteLine("MyMiddleware1: Outgoing Response");
    await context.Response.WriteAsync("MyMiddleware1: Outgoing Response\n");
});
app.Run(async (context) =>
{
    //Write the message to the response
    var response = "MyMiddleware2: Terminating Request\n";
    Console.WriteLine("MyMiddleware2: Terminating Request");
    await context.Response.WriteAsync("MyMiddleware2: Terminating Response\n");
});
//Run the application
app.Run();
```

Save the file and run the application, and you should get the following output in the console.



Understanding ASP.NET Core Request Processing Pipeline Execution Order:

Let us compare the above output with the following diagram to understand the ASP.NET Core Request Processing Pipeline Execution.



Now, when the incoming HTTP request arrives, it is first received by the first middleware component, Middleware1, which logs the request and passes it to the next middleware. The request then goes through Middleware2, which processes the request and passes it to the final middleware, Middleware3. In the last step, instead of calling the next middleware, Middleware3 generates the response and starts processing the response pipeline. Middleware2 processes the response and passes it to Middleware1, which logs the response and sends it back to the client.

When the incoming HTTP request arrives, it is first received by the first middleware component, Middleware1, which logs the request and passes it to the next middleware. The request then goes through Middleware2, which processes the request and passes it to the final middleware, Middleware3. In the last step, instead of calling the next middleware, Middleware3 generates the response and starts processing the response pipeline. Middleware2 processes the response and passes it to Middleware1, which logs the response and sends it back to the client.

The first middleware handles the request and then processes the response. So, the first information that we see is the message **1 Middleware1: Incoming Request** and **response generated**.

The third middleware component is registered using the Run() Extension Method and is a terminal middleware component. It does not call the next middleware in the pipeline. So, the message **3 Middleware3: Terminating Request** is printed in the console first. Then, the middleware **Middleware3: Terminating Response** is written to the response. The last middleware component is the first middleware component, and the first middleware component logs the information as **Middleware1: Outgoing Response** and sent to the browser.

Key Points to Remember:

- The ASP.NET Core Request Processing Pipeline consists of a sequence of Middleware Components that will be called one after the other.
- Each middleware component can perform some operations before and after invoking the next Middleware Component using the next() method. A middleware component can also decide not to call the next middleware component and stop the pipeline called short-circuiting the request pipeline.
- The Middleware Components in ASP.NET Core are access in both the incoming request and the outgoing response.

The main important point to keep in the mind is which the Middleware Components are added in the Main method of the Program class define the order in which these Middleware Components will be invoked on incoming requests and in the reverse order for the response. So, the order is crucial for defining the application's security, performance, and functionality.

In the next article, I will discuss the Custom Order in ASP.NET Core Application. In this article, I try to explain the ASP.NET Core Request Processing Pipeline with an example. I hope you enjoy this article.

Author: Pranay Rana
Pranay Rana is a Microsoft MVP, Author, Speaker, Trainer, Consultant and Blogger. He has 12 years of experience in Microsoft .NET technologies. He is the author of ASP.NET Web API 2 Step by Step, ASP.NET Web API 2 Recipes, ASP.NET MVC 5 Step by Step, ASP.NET MVC 5 Recipes, ASP.NET MVC 6 Step by Step, ASP.NET MVC 6 Recipes, ASP.NET Web API Security Essentials, Mastering Microsoft Azure Development, and Learning Microsoft Azure Development.

Previous Lesson: ASP.NET Core Middleware Components
Next Lesson: enumerate Folders in ASP.NET Core

4 thoughts on "ASP.NET Core Request Processing Pipeline"

Reply

Sona
December 1, 2024 at 12:29 PM

Very an extensive info. Thanks!

Reply

CHITHANDAN
December 2, 2024 at 12:12 PM

great article , helps me a thanks

Reply

MOHAMMAD K
December 2, 2024 at 12:13 PM

Thanks for sharing information, Great article

Reply

LAKHAN
January 7, 2025 at 3:49 PM

Sir, I have a doubt:

In the pipeline, you mentioned HTTS (presume, meaning HTTP redirect) and then comes HTTP redirection. Shouldn't the order be reversed? Because, if a request comes in as HTTP, it should first be redirected to HTTPS, then the authentication, etc. If the first middleware component is HTTS, wouldn't it be bypassed if the request is HTTP? Or, do HTTS middleware also handle HTTP redirection, and the rest (authentication, etc.) only run if the request is HTTPS, as the requests are redirected to HTTPS anyway?

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment